

Flipping bits

You will be given a list of 32 bit unsigned integers. Flip all the bits (1 -> 0 and 0 -> 1) and return the result as an unsigned integer.

The core task is trivial: we need to change every 0 to a 1 and every 1 to a 0. As we covered in Level 1, this is the exact definition of the bitwise NOT operator (~).

However, the trap lies in the 32nd bit.

The Signed vs. Unsigned Divide:

In standard CP environments, a regular int is a 32-bit signed integer. It uses Two's Complement representation, meaning the leftmost bit (the Most Significant Bit, or MSB) is reserved as the sign bit.

- If the MSB is 0, the number is positive.
- If the MSB is 1, the number is negative.

Let's say the input is 1 (which is thirty-one 0s followed by a 1). If you apply ~1 on a signed int:

1. All bits flip. The memory now holds thirty-one 1s followed by a 0.
2. Because the MSB is now 1, the CPU interprets this memory block as a negative number.
3. It prints -2.

But the problem explicitly demands a 32-bit unsigned integer. An unsigned int does not have a sign bit. It uses all 32 bits purely for magnitude, giving it a range of 0 to 4294967295 ($2^{32} - 1$).

If you treat that exact same block of memory (thirty-one 1s followed by a 0) as an unsigned int, the CPU translates it to 4294967294\$.

Code

```
// Using uint32_t guarantees an unsigned 32-bit integer strictly
uint32_t flippingBits(uint32_t n) {
    // The ~ operator flips all 32 bits.
    // Because the return type is uint32_t, it will not be evaluated as a negative number.
    return ~n;

    // Alternative approach using XOR:
    // return n ^ 0xFFFFFFFF;
}
```

Reverse Bits

Reverse bits of a given 32 bits signed integer.

Integer	Binary
43261596	00000010100101000001111010011100
964176192	00111001011110000010100101000000

The Bit-by-Bit Algorithm:

1. Create a result variable initialized to 0.
2. Loop exactly 32 times (since we are dealing with a strict 32-bit boundary).
3. Extract: Grab the rightmost bit of n using n & 1.
4. Make Room: Shift result to the left by 1 (result << 1) to make space for the incoming bit.
5. Insert: Append the extracted bit to result using the OR operator (result | bit).
6. Shift Down: Shift n to the right by 1 (n >> 1) so the next bit is ready to be extracted on the next loop.

Code

```
int reverseBits(int n) {
    int result = 0;

    for (int i = 0; i < 32; i++) {
        // Shift result to the left to make room for the new bit
        result <<= 1;

        // Extract the rightmost bit of n and add it to result
        result |= (n & 1);

        // Shift n to the right to process the next bit
        n >>= 1;
    }

    return result;
}
```